

Notes on Abstract Data Types, Sorting Algorithms and Discrete Structures

Edward Chernysh

1 Sorting Algorithms

```
int [] insertionSort(int [] input) {
    int temp = 0, j = 0;
    if (input == null || input.length <= 1)
        return input;

    for (int i = 1; i < input.length; i++) {
        temp = input[i];
        j = i;
        while (j > 0 && temp < input[j-1]) {
            int alt = input[j-1];
            input[j-1] = temp;
            input[j] = alt;
            j--;
        }
    }
    return input;
}
```

We consider the *MergeSort* algorithm. This algorithm operates via divide-and-conquer. Generally, this algorithm will take an array A divide it in half, recursively sort each half and merge them. Let $T(n)$ be the function describing run-time complexity of merge sort. Note that

$$T(n) = \begin{cases} 0 & n = 1 \\ 2T(\frac{n}{2}) + n & n > 1 \end{cases}$$

Thus $T(n) \in \mathcal{O}(n \log n)$. We will be able to prove this elegantly once we develop the *Master Theorem* below.

2 The Master Theorem

The master theorem is used to solve many recurrence relations. That is, equation of the form

$$T(n) = aT\left(\frac{n}{b}\right) + f(n)$$

Where the constants $a \geq 1, b > 1$ and $f > 0$ are the number of sub-instances, number of divisions and merge-time respectively. The cases are as follows:

Case #	Condition	Solution
(1)	$f(n) \in \mathcal{O}(n^L)$ where $L < \log_b a$	$T(n) \in \Theta(n^{\log_b a})$
(2)	$f(n) \in \Theta(n^{\log_b a} \log^k n)$ for $k \geq 0$	$T(n) \in \Theta(n^{\log_b a} \log^{k+1} n)$
(3)	$f(n) \in \Omega(n^L)$ for $L > \log_b a$ and $f(n)$ is <i>regular</i>	$T(n) \in \Theta(f(n))$

Regularity: A function $f : \mathbb{N} \rightarrow \mathbb{R}$ is said to be regular if there exists $c < 1$ and $N \in \mathbb{N}$ such that for all $n \geq N$ we have $af(n/b) \leq cf(n)$

Let us now consider some examples of problems to which these may be applied.

1. $T(n) = 5T(n/2) + \Theta(n^2)$. Here all we know is $f(n) \in \Theta(n^2)$. Thankfully this is sufficient to solve the relation. Note that $L2 < \log_b a = \log_2 5 \approx 2.25$. Thus, Case 1 applies here. Hence, we have by the table above that $T(n) \in \Theta(n^{\lg 5})$
2. $T(n) = 27T(n/3) + \Theta(n^3 \log n)$. In this case, $\log_b a = \log_3 27 = 3$. Indeed, $f(n) \in \Theta(n^{\log_b a} \log^k n)$ where $k = 1$. Hence, by case 2 it follows that

$$T(n) \in \Theta(n^3 \log^2 n)$$

3. $T(n) = 5T(n/2) + \Theta(n^3)$. In this case $\log_b a = \log_2 5 \approx 2.25 < L = 3$. Indeed, $f(n) \in \Omega(n^3)$ and hence $L > \log_2 5$. We only need to prove regularity. Consider:

$$af(n/b) = 5 \left(\frac{n}{2}\right)^3 = \frac{5}{8}n^3$$

Taking $c = 5/8 < 1$ and $n = 1$ we have regularity. Hence, by case (3) of the master theorem it follows that

$$T(n) \in \Theta(n^3)$$

Divide and Conquer Paradigms

This approach to problem solving involves breaking the problem apart into smaller pieces, recursively solving each half and recombining. More often than not, one will divide the problem into two sub-pieces each recursion. Let us now consider the *Binary Search Algorithm*. This algorithm takes as input the following quadruple: $(A, v, low, high)$ where A is a sorted array of integers, $v \in \mathbb{Z}$ the value we are searching for and low and $high$ the incides of the array being considered. We will break this problem apart into sub-problems as below:

```

int binarySearch(int [] A, int v, int low, int high) {
    if (high == low)
        if (A[low] == v)
            return low;
        else return (-1);

    int breakpoint = (int)(high-low)/2;
    if (v <= A[breakpoint])
        return binarySearch(A, v, low, breakpoint);
    else
        return binarySearch(A, v, breakpoint + 1, high);
}

```

This algorithm will break down the problem until a singleton is left, at this point it will check and see whether or not this $\{v\} = v$. It will either return the position in A or an EOF marker to signal that no such element can be found in the array. Until that point, it will divide the array into two half (these halves may differ by 1). It will check the end point of the lower half and determine whether the element can be found in it. If not we discard this and explore the other half recursively. It is important to note that this is only made possible by the fact that the array is already sorted.

We offer a brief analysis of this search algorithm. In the case where A is a singleton, it takes constant time since we only need to check if this element corresponds to the target of the search.

Otherwise, we divide it into two halves, but make 1 comparison to decide which half to explore. Hence, this results in the following recursion

$$T(n) = \begin{cases} 1 & n = 1 \\ T\left(\frac{n}{2}\right) + 1 & n > 1 \end{cases}$$

This corresponds the second case of the Master Theorem, to see this note that $\log_b a = \log_2 1 = 0$. Hence $n^0 \log^0 n = 1$ and hence the criterion for case (2) is satisfied. Thus, $T(n) \in \Theta(\log n)$.

3 Linked Lists

We begin by defining a node class that will serve as a storage device for the objects in question. Most of all, they will contain pointers to other nodes that serve as the structural component of these lists. It is important to note that the node class will vary with data types, for instance a node used in a DLinkedList will have one extra pointer than that used in an SLinkedList. We will present and distinguish these types of nodes as we proceed. For now, consider the following data type:

```
// Node for SLinked List
class Node<T> {
    Node next;
    T object;
}
```

Having defined this object, it is easy to implement a *Singly Linked List* by defining the following:

```
class SLinkedList<T> {
    Node head, tail;
    int size;
}
```

Obviously you can define a constructor that best suits the situation you are working with so we will neglect this in this text. Moreover, storage of the nodes is best done in an array like structure, although this not *required*, since you could generate new nodes dynamically throughout your algorithm and have them connected only by the pointers within the nodes. Given this structure we can define some algorithms that handle `addFirst()`, `removeFirst()`, `addLast()`, `removeLast()` described below, where we assume the given methods live within the class as public methods that can access the Object:

```
void addFirst(Node<T> x) {
    x.next = this.head;
    this.head = x;
    (this.size)++;
}

Node<T> removeFirst() {
    Node<T> first = this.head;
    this.head = this.head.next;
    first.next = null;
    return first;
    --(this.size);
}

void addLast(Node<T> x) {
    this.tail.next = x;
```

```

    this.tail = this.tail.next;
    this.size++;

}

Node<T> removeLast() {
    Node<T> last = this.tail;
    if (this.head == this.tail) {
        this.head = null;
        this.size = 0;
        return last;
    }

    Node<T> ptr = this.head;
    while(ptr.next != this.tail)
        ptr = ptr.next;
    ptr.next=null;
    this.tail = ptr;
    (this.size)--;

}

```

We extend these concepts towards *doubly linked lists* which we hereby refer to as DLinkedLists. We implement the same methods, however we modify the node as follows:

```

class Node<T> {
    T object;
    Node<T> previous, next;
}

```

The list structure remains identical:

```

class DLinkedList<T> {
    int size;
    Node<T> head, Node<T> tail;
}

```

Under these new axioms we consider the same methods in this context:

```

Node<T> getNode(int index) {
    if (i < size/2) {
        Node<T> ptr = this.head;
        for (int i= 0; i < index; i++)
            ptr = ptr.next;
        return ptr;
    }
    else {
        Node<T> ptr = this.tail;
        int pos = this.size -1;
        while (pos > i) {
            ptr = ptr.previous;
            i--;
        }
        return ptr;
    }
}

```

```

}

Node<T> remove (Node<T> node) {
    node.next.previous = node.previous;
    node.previous.next = node.next;
    (this.size)--;
    return node;
}

```

In general, methods will follow this structure. Another important question arises...how should one implement this on a machine? Again, as stated previously it is possible to simply have *free* nodes in space, linked to each other only by the pointers they each carry. In fact, this is my preferred approach. Unfortunately, some claim this is a bad idea. Often one creates an array with a fixed ceiling for size and sets all positions to be **null**, and will use this as a container for the nodes themselves. Often an ArrayList will be more useful, particularly when the task at hand may require frequently changing the *value* of elements at known positions.

4 Stacks and Queues

We will consider two very useful abstract data types that are used when searching discrete graphs and structures. We will see applications of these later in the following section. We now define a **stack**.

Definition. A **stack** is a container that stores objects together with the following:

- Objects can be pushed(added) to the top at any time.
- Only the top of the stack (the object most recently added) can be popped (removed).
- These two operations are denoted $push(o)$, $pop()$. Moreover, many implementations carry $size()$, $isEmpty()$ and a $top()$ method which peaks at the top without removing the object.

Consider the following string: $3 + (4 - 1) * 7 + (6 - 2 * (2 + 3))$. One can use a stack to evaluate this string whilst respecting the conventions of arithmetic. To see this, we describe the stack in question as follows

```

      + +
    ( ( ( (
      * * * * *
    - - - - -
  ( ( ( ( ( ( ( ( ( (
+ + + + + + + + + + + -

```

Now we proceed to examine another abstract structure called a Queue. This operates in a similar, but antagonistic way to the stack. We give a crucial definition below:

Definition. Let $\mathcal{O}$ be a collection of objects and $\sigma \in \mathcal{O}$. A Queue is an abstract structure equipped with two fundamental operations and 3 defined methods:

- $enqueue(\sigma)$ - which adds the object to the back of the underlying structure.
- $dequeue$ - which "pops" the element off the top of the container.
- Methods $size()$, $isEmpty()$ and $front()$ which allows peeking as before.

Remark: One should take note that adding elements to the "back" and popping elements off the "front" implies that we can only grab the element that has spent the most time in our list. Hence, priority is given to older objects. Hence the name *Queue*.

5 Induction and Recursion

The principle of mathematical induction relies on a crucial axiom. Although, this axiom should be intuitively obvious. This is a consequence of the fact that $\mathbb{N}$ is closed and bounded below.

Axiom (Well Ordering Axiom). *Any non-empty subset of $\mathbb{N}$ has a least element.*

With this in mind, we can now construct (or more appropriately *prove*) a crucial tool for mathematics and computer science. This will be extremely useful when dealing with recurrence relations.

The Principle of Mathematical Induction. *Let P_i be a propositional statement for $i \in \mathbb{N}$. Suppose further:*

1. P_1 is true.
2. P_n true implies P_{n+1} is true for all $n \in \mathbb{N}$.

Then P_n is true for all $n \in \mathbb{N}$.

Proof. We argue by contradiction. Suppose that P_n is not true for all $n \in \mathbb{N}$, then P_N is false for some $N \in \mathbb{N}$. Moreover, by hypothesis that P_1 is true, we have that $N > 1$. Define now the set

$$\mathcal{P} := \{n \in \mathbb{N} \mid P_n \text{ is false}\}$$

Indeed, $N \in \mathcal{P}$ by assumption and hence $\mathcal{P}$ is a non-empty subset of $\mathbb{N}$. It then follows from the *Well Ordering Axiom* that $\mathcal{P}$ has a least element $m \in \mathbb{N}$. That is, for all $n < m$ we have that P_n holds true. In particular, $m - 1 < m$ implies P_{m-1} is true. By (2) this implies P_m is true. Contradiction. $\square$

Example. Show that for $x > -1$ one has for all $n \in \mathbb{N}$ that: $(1 + x)^n \geq 1 + nx$. This is known as *Bernoulli's Inequality*.

Proof. For $n = 1$ we have $(1 + x)^1 = 1 + x = 1 + 1 \cdot x$ which is obviously true. By way of induction suppose for $n \in \mathbb{N}$ we have $(1 + x)^n \geq 1 + nx$, we will show this implies $(1 + x)^{n+1} \geq 1 + (n + 1)x$. Consider:

$$\begin{aligned} (1 + x)^{n+1} &= (1 + x)(1 + x)^n \geq (1 + x)(1 + nx) = 1 + x + nx + nx^2 \\ &= 1 + (n + 1)x + \underbrace{nx^2}_{\geq 0} \geq 1 + (n + 1)x \end{aligned}$$

$\square$

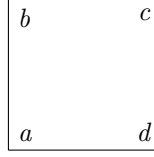
6 Discrete Structures

Definitions

Definition. *A graph $\mathcal{G}$ of elements in X is a subset of X denoted $\mathcal{G}$ is a set V of points in X together with a set E of ordered pairs connecting each vertex in V . If this is the case, then we write $\mathcal{G} = (V, E)$.*

Remark: an edge $e \in E$ is a pair $(u, v) \in V \times V$.

Example 1. *Let $V = a, b, c, d$ and define $E := \{(a, b), (b, c), (c, d), (d, a)\}$ describes a square:*



This discrete structure obviously has many applications, notably in transportation, communication, WWW, software, circuits and schedules.

Terminology

- adjacent vertices are connected by an edge.
- For a vertex $v \in V$, the degree $\mathcal{D}$ of v denoted $\mathcal{D}(v) :=$ number of adjacent vertices.

Proposition 1. Let $\mathcal{E}$ denote the number of edges of a graph $\mathcal{G} = (V, E)$. Then

$$\sum_{v \in V} \mathcal{D}(v) \equiv 2 \cdot \mathcal{E}$$

Proof. Let $\alpha, \beta \in V$ be two distinct but arbitrary vertices. Then by definition they are connected by some edge $\eta \in E$. Consider the vertex α , then $\mathcal{D}(\alpha) = \sum_{\text{edges } e \text{ of } \alpha} e$. Similarly, $\mathcal{D}(\beta) = \sum_{\text{edges } \tilde{e} \text{ of } \beta} \tilde{e}$. Now each sum will include η and thus η will be counted exactly twice. To see this, suppose instead it is counted $n > 2$ times. Then it must connect $n > 2$ nodes which contradicts the structure of a graph.

Hence since we chose α, β arbitrarily we conclude $\sum_{v \in V} \mathcal{D}(v) \equiv 2 \cdot \mathcal{E}$. $\square$

Definition. Let $\mathcal{G} = (V, E)$ be a graph. A path is a sequence of vertices $\{v_1, v_2, \dots, v_k\} \subseteq V$ such that for any $i \in [1, k] \cap \mathbb{N}$, the vertices $v_i \wedge v_{i+1}$ are adjacent.

Moreover, if each element in $\{v_1, v_2, \dots, v_k\}$ is distinct (i.e not repeated) then we call the path simple.

Definition. A cycle is a simple path with the added condition that the final vertex is the first vertex.

Definition. A connected graph is a graph $\mathcal{G} = (V, E)$ such that for any two vertices $u, v \in V$ there exists a path $\mathcal{P}$ connecting u and v .

Whence we define a subgraph as a subset of the original graph. So, given a graph $\mathcal{G} = (V, E)$ we could define a subgraph $\mathcal{Q} = (W, F) \subset \mathcal{G}$. A connected component is simply a connected subgraph $\mathcal{Q}$ of $\mathcal{G}$.

Definition. A (free) tree is a connected graph **without** cycles. A forest is a collection of trees.

Connectivity

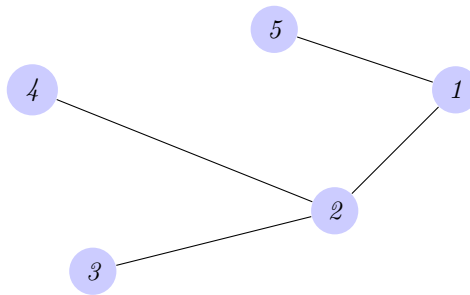
We say a graph is complete if all pairs of vertices are adjacent.

Proposition 2. Let $\mathcal{G}$ be a complete graph and $n = \#$ of vertices and $m = \#$ of edges of a graph $\mathcal{G} = (V, E)$. Then

$$m = \frac{1}{2} \sum_{v \in V} \mathcal{D}(v) = \frac{1}{2} \sum_{v \in V} (n-1) = \frac{n(n-1)}{2} \quad (1)$$

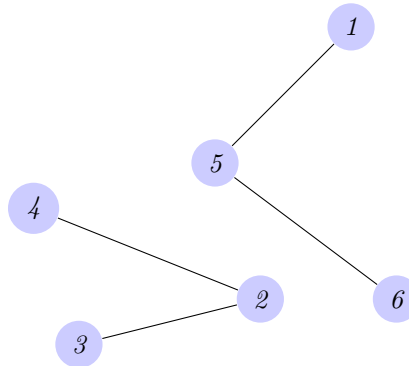
Proof. The proof is given in the statement, I simply wish to make a remark. This relation holds because each of the n vertices is incident to exactly $n-1$ edges! $\square$

Example 2. For a tree $m = n - 1$



$$n = 5, m = 4$$

Example 3. If $m < n - 1$ then the graph $\mathcal{G}$ is not-connected.



$$n = 6, m = 4 \implies 4 = m < n - 1 = 5$$

Spanning Trees

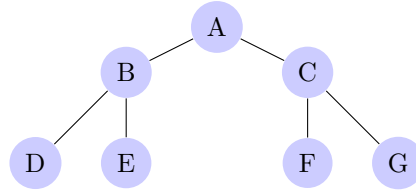
Definition. A spanning tree $\mathcal{G}$ is a subgraph which

- is a tree
- contains all vertices of $\mathcal{G}$.

Let us now discuss some properties of a graph container (or some similar ADT). Let $\mathcal{G} = (V, E)$ be a graph and let $v, w \in V$ and $e \in E$ and o be some object.

Method	Return	Method	Return
<code>size()</code>	# vertices + # edges of $\mathcal{G}$	<code>isEmpty()</code>	boolean
<code>elements()</code>	# vertices ?	<code>positions()</code>	??
<code>swap()</code>	??	<code>replaceElement()</code>	??
<code>numVertices()</code>	returns # of vertices of $\mathcal{G}$	<code>numEdges()</code>	# of edges
<code>vertices()</code>	enum of vertices	<code>edges()</code>	enum of edges
<code>directedEdges()</code>	enum of all directed edges	<code>undirectedEdges()</code>	$\oint$
<code>incidentEdges(v)</code>	enum of edges incident to v	<code>inIncidentEdges(v)</code>	incoming edges to v
<code>outIncidentEdges(v)</code>	enum of outgoing edges from v	<code>opposite(v,e)</code>	Endpoint of $e \neq v$
<code>degree(v)</code>	$\mathcal{D}(v)$	<code>inDegree(v)</code>	outDegree of v
<code>outDegree(v)</code>	out-degree	<code>adjacentVertices(v)</code>	adjacent vertices to v
<code>inAdjacent(v)</code>	$\{w\}$ along incoming	<code>outAdjacentVertices(v)</code>	""

A tree represents a hierarchy. Examples are the UNIX/DOS filesystem. Now, some terminology. Consider the following tree:



- A is the root node and B is the parents of D and E .
- C is the sibling of B . D, E, F, G are external-nodes/leaves.
- the depth of E is 2.
- The height of the tree is 2.
- the Degree of node B is 2.

Binary Trees

Definition. (Recursive definition of B-tree) A binary tree is either an external node (leaf) or an internal node (root) with two (left and right) binary sub-trees.

Example 4. Arithmetic expression or decision tree.

Here are some simple properties of a binary tree:

- Nodes have ordered children.
- ALL internal nodes have a degree of 2.
- # of external nodes = # internal nodes + 1
- # of nodes at level $i \leq 2^i$. Where $0 \leftrightarrow \text{root}$ ($2^0 = 1$).
- # of external nodes $\leq 2^{\text{height}}$
- $\text{height} \geq \log_2 (\# \text{ external nodes})$
- $\text{height} \geq \log_2 (\# \text{ nodes} - 1)$
- $\text{height} \leq \# \text{ internal nodes} = (\# \text{ nodes} - 1)/2$

6.1 Searching Through Trees

Depending on the tree, many different searching algorithms are possible. Many are based on two basic and fundamental algorithms, called *Depth-First-Search* and *Breadth-First-Search* appropriately. The former will explore the maze while tagging previously visited nodes, going down paths until the desired element is found or the end of the path is reached. The latter will instead check nodes at the start of each path whilst expanding outwards. The algorithms are given below:

```

class Node<T> overrides Node<T> {
    boolean tagged;
}

boolean depthFirstSearch(Node<T> start, T search) {
    if (start.equals(search))
        return true
    else {
        Note<T>[] children = start.children;
        boolean found = false;
        for (int i = 0; i < children.length; i++) {
            if (!children[i].tagged)
                found |= depthFirstSearch(children[i], search);
            children[i].tagged = true;
        }
        return found;
    }
}

boolean breadthFirstSearch(Node<T> start, T search) {
    if (start.equals(search))
        return true;
    Queue<T> queue = new Queue<T>(start);
    Node<T> top = new Node<T>();
    while (!queue.isEmpty()) {
        top = queue.pop();
        top.tagged = true;
        if (top.equals(search))
            return true
        for (Node<T> n : top.children)
            if (n.tagged == false)
                queue.pop(n);
    }
    return false;
}

```

Equipped with these new concepts we will move onto Binary Search Trees and their algorithms.

Definition. A binary search tree is a tree such that the left children of a node all have keys less than or equal to that node, and the right children all have keys greater than or equal to it.

Consider a tree containing a subset of some orderable $\mathcal{T}$. We consider an algorithm that searches for some $z \in \mathcal{T}$ in this tree. The set have order operations is absolutely crucial, since otherwise one would be unable to construct a binary search tree from the set, as each node must have a numerical key associated to the item. Moreover, we must be able to make comparisons to determine where we should continue the search. In fact, this is what will allow us to search for an item τ in $\mathcal{O}(\log n)$ time.

```

boolean bstContains(Node<T> root, T target) {
    if (root.key == target.key)
        return true;
    else if (root.children == null)
        return false;

    else {
        if (root.key > target.key)
            return bstContains(root.leftChild(), target);
        else if (root.key < target.key)
            return bstContains(root.rightChild(), target);
    }
}

Node<T> bstSearch(Node<T> root, T target) {
    if (root.key == target.key)
        return root;
    else {
        if (root.children == null)
            return root; // this is an external node
        else if (root.key > target.key)
            return bstSearch(root.leftChild(), target);
        else if (root.key < target.key)
            return bstSearch(root.RightChild(), target);
    }
}

```

Insertion is somewhat less straightforward, consider a pair (e, k) where k is the key of some object e . Let $w = \text{bstSearch}(\text{root}, e)$. If w is external, we can simply set it to be (e, k) and generate empty children. However, if it is an internal node, then this implies the existence of some other node in the graph with the same key. In this case we need only call the insertion algorithm recursively on either child of this node, supposing each child is not filled.

A problem that arises naturally is that of removal. How should one go about removal? Naturally, we locate the node using the search algorithm described above. Wlog we suppose a node is found. If this node